

Chapter 2

Operating-System Structures

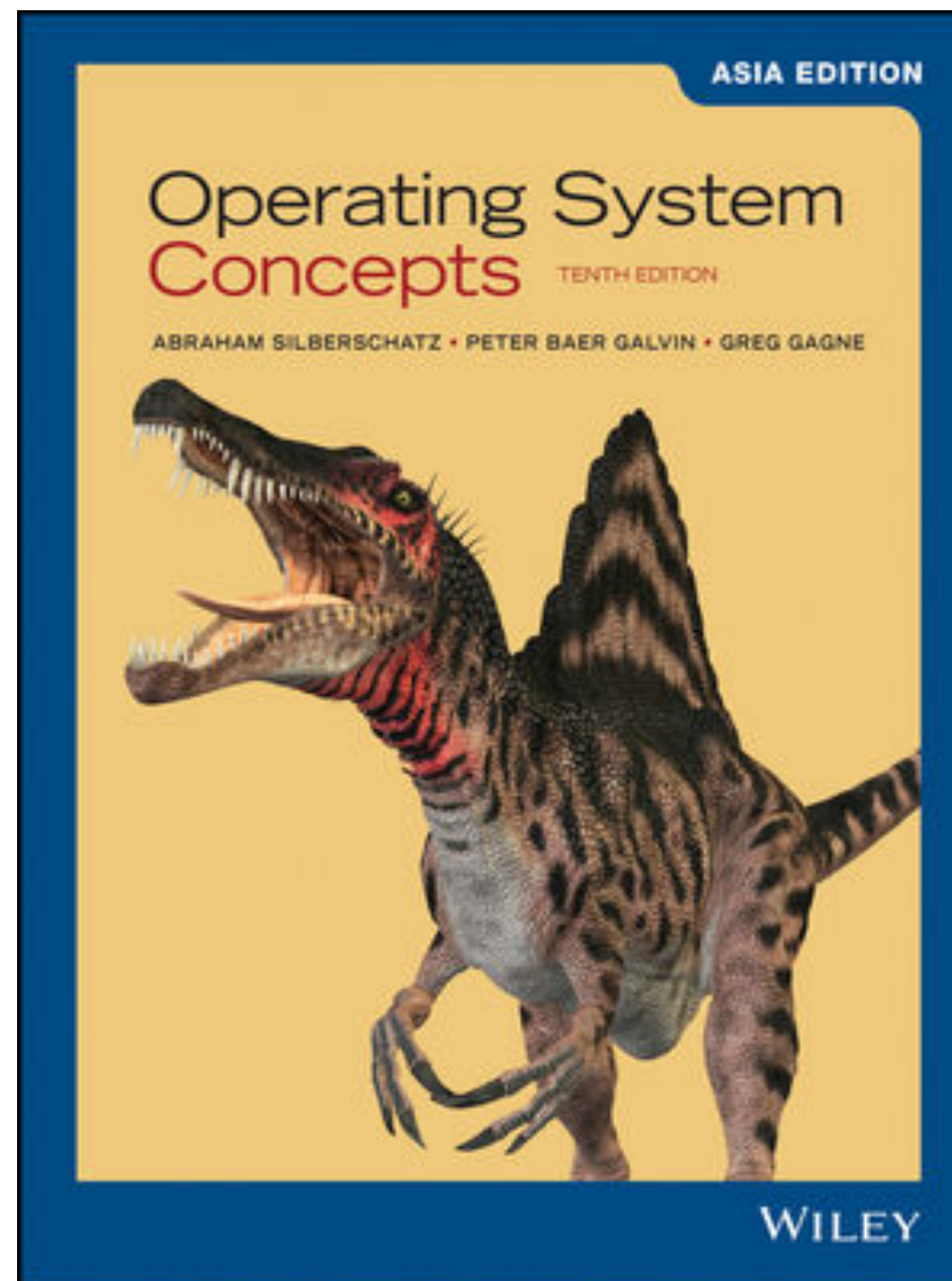
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/7/3

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
 - **User interface** - Almost all operating systems have a user interface (UI).
 - Varies between **Command-Line** (CLI), **Graphics User Interface** (GUI), **touch-screen**
 - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
 - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

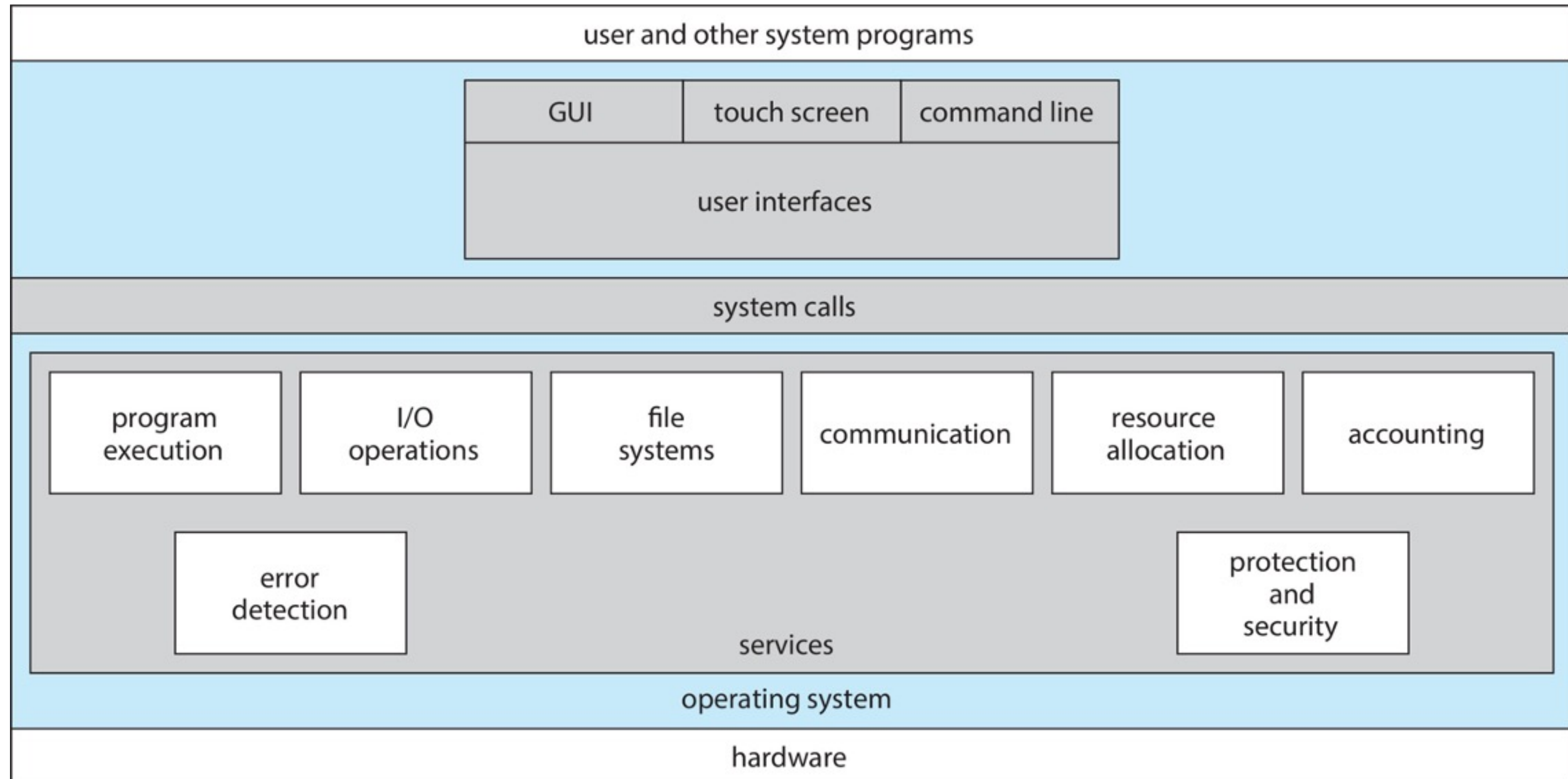
Operating System Services

- One set of operating-system services provides functions that are helpful to the user:
(Cont.)
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
- Communications may be via shared memory or through message passing (packets moved by the OS)

Operating System Services

- Another set of operating-system functions exists not for helping the user but rather for ensuring the **efficient operation of the system itself**
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
 - **Logging** - To keep track of which users use how much and what kinds of computer resources
 - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - **Protection** involves ensuring that all access to system resources is controlled
 - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

A View of Operating System Services



Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

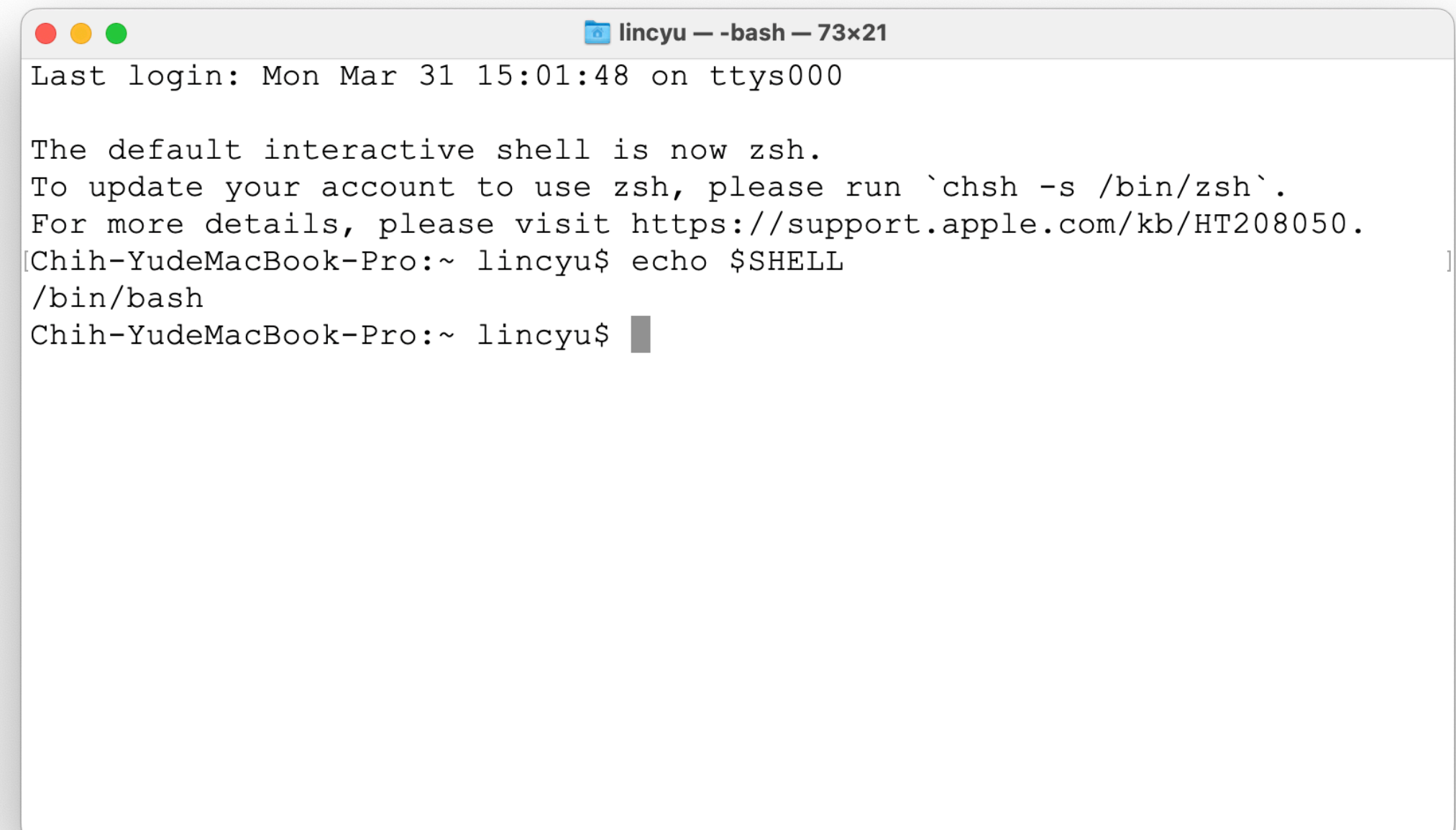
Command Interpreters

- On systems with multiple command interpreters to choose from, the interpreters are known as **shells**
- C shell, Bourne-Again shell, Korn shell, **PowerShell**
- auto-completion (**tab**)



```
Slide — -zsh — 68x19

lincyu@Chih-YudeMacBook-Pro Slide % pwd
/Users/lincyu/Desktop/Data/NTOU (2025)/1141課程/OS/Slide
lincyu@Chih-YudeMacBook-Pro Slide % echo $SHELL
/bin/zsh
lincyu@Chih-YudeMacBook-Pro Slide %
```



```
lincyu — -bash — 73x21

Last login: Mon Mar 31 15:01:48 on ttys000

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
Chih-YudeMacBook-Pro:~ lincyu$ echo $SHELL
/bin/bash
Chih-YudeMacBook-Pro:~ lincyu$ █
```

Graphical User Interface

- User-friendly **desktop** metaphor interface
 - Desktop metaphor: https://en.wikipedia.org/wiki/Desktop_metaphor
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
 - Invented at Xerox PARC
 - <https://crm.org/articles/xerox-parc-and-the-origins-of-gui>
- Many systems now include both CLI and GUI interfaces

Touch-Screen Interface

- Touchscreen devices require new interfaces
 - Mouse not possible or **not desired**
 - Actions and selection based on **gestures**
 - Virtual keyboard for text entry
- Voice commands



Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

System Calls

- **System calls** provide an interface to the services made available by an operating system
- These calls are generally available as **functions** written in C and C++

CL Is `printf` in C a system call? If not, does `printf` function call system call? If so, what system calls are called by `printf`?

No, `printf` in C is **not** a system call. It's a **library function** provided by the C standard library (libc).

Does `printf` call system calls?

Yes, `printf` eventually calls system calls to perform the actual output operation.

System calls used by `printf`

The specific system calls depend on the operating system:

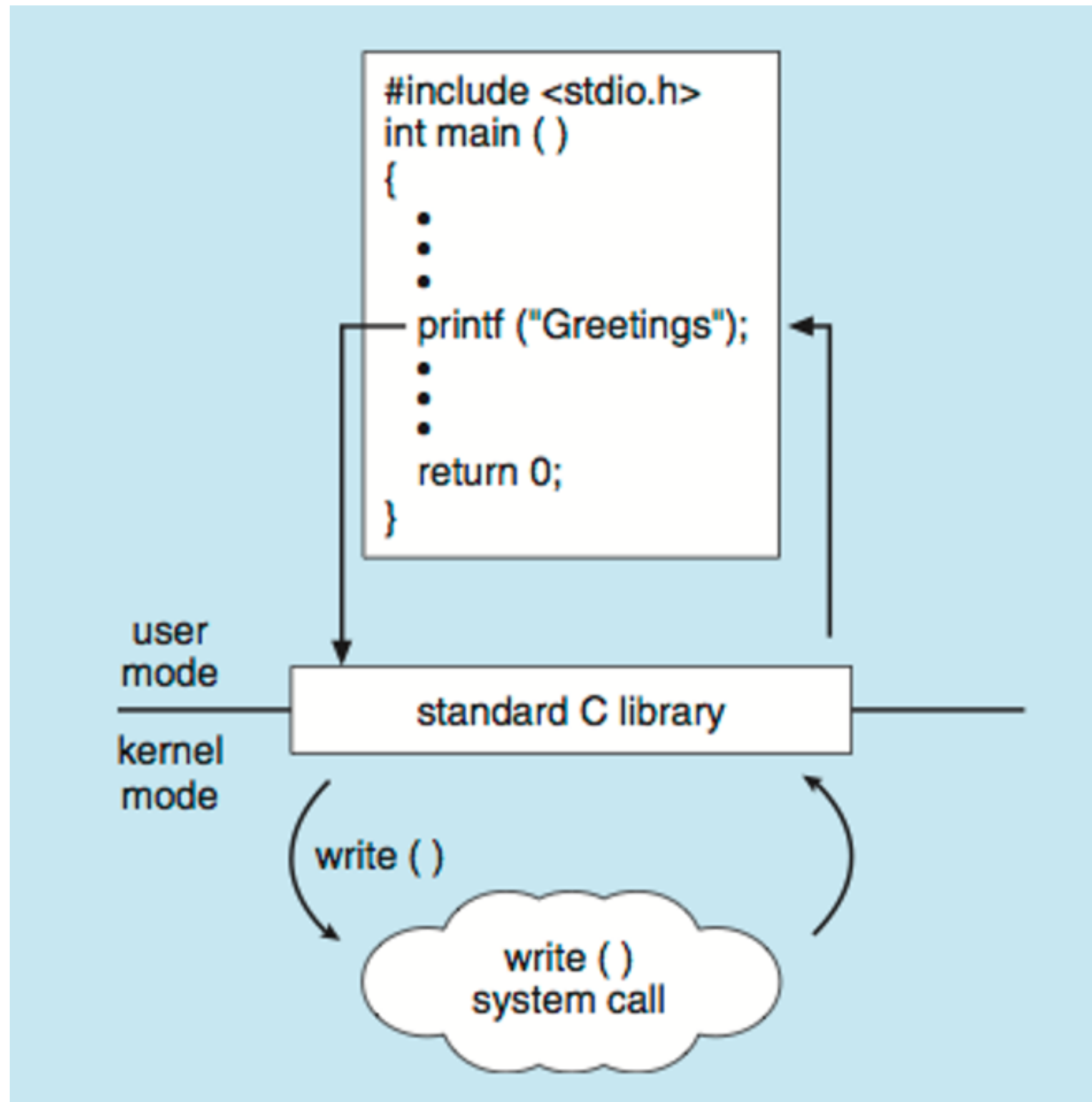
On Linux/Unix systems:

Primary system call: `write()`

C

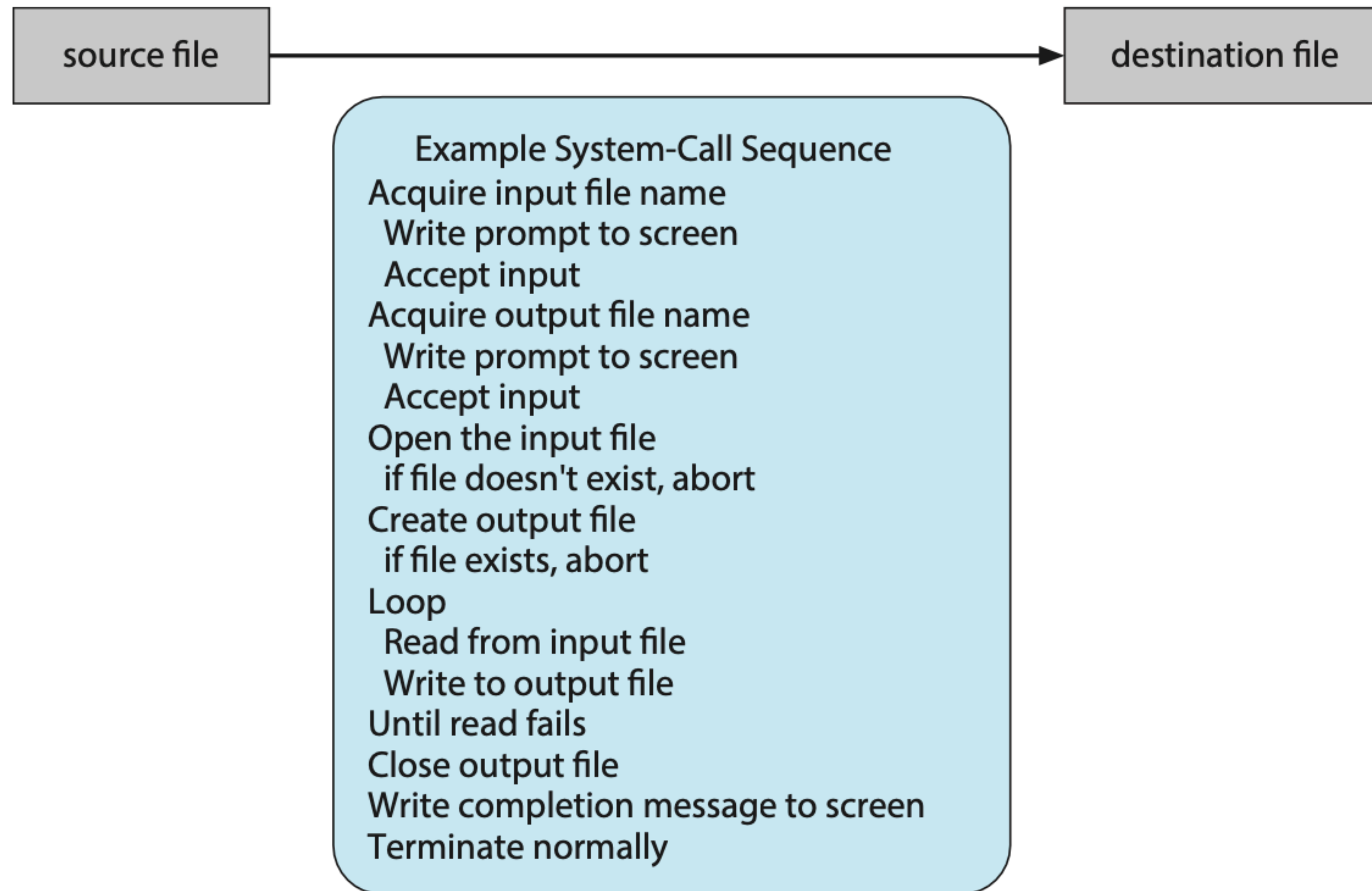
```
write(file_descriptor, buffer, count)
```


The Standard C Library



```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <fcntl.h>
4  #include <sys/types.h>
5  #include <sys/stat.h>
6
7  int main() {
8      // write() system call wrapper
9      write(1, "Hello World\n", 12);
10
11     // getpid() system call wrapper
12     printf("Process ID: %d\n", getpid());
13     return 0;
14 }
```

Example: cp in.txt out.txt

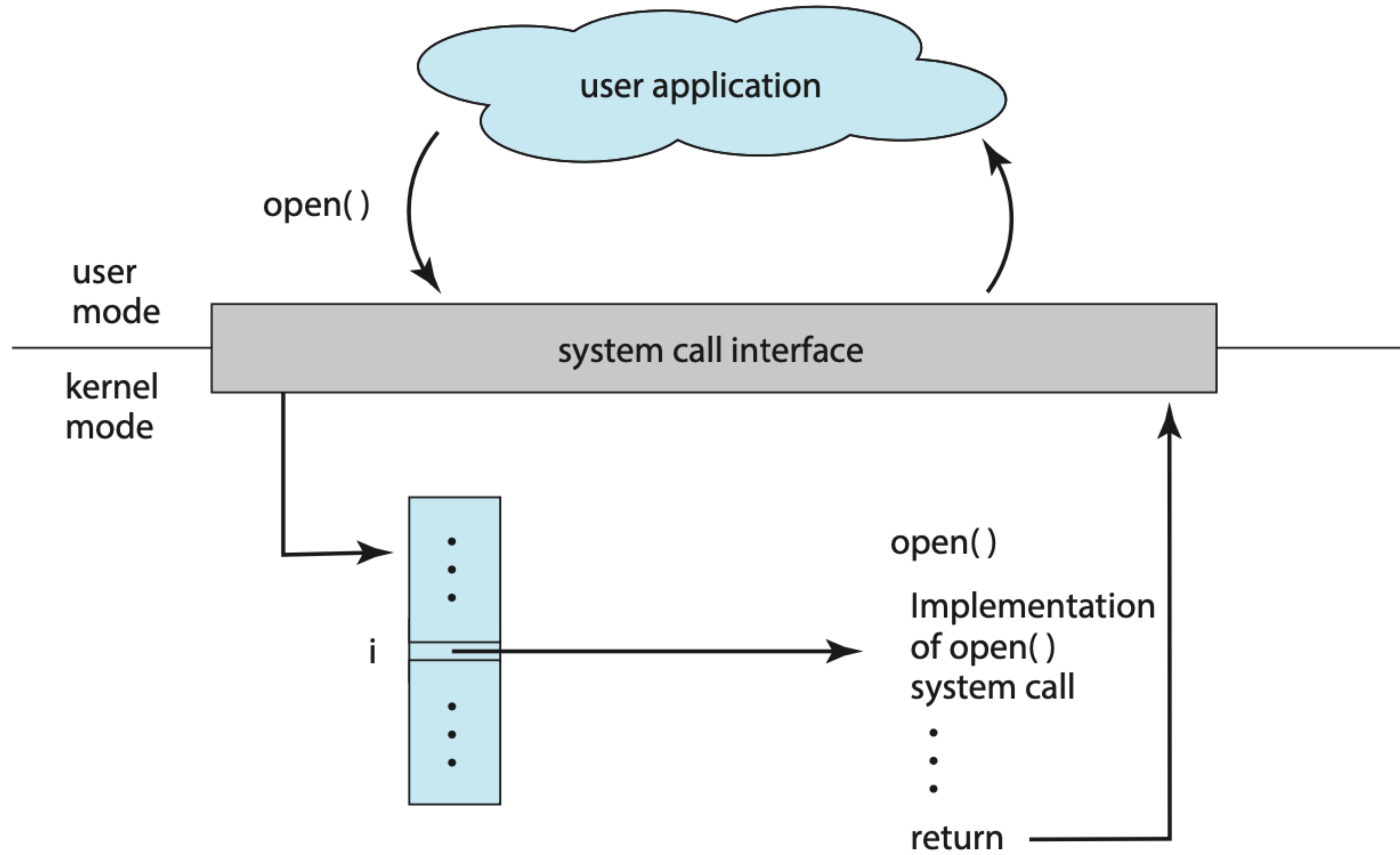


Application Programming Interface

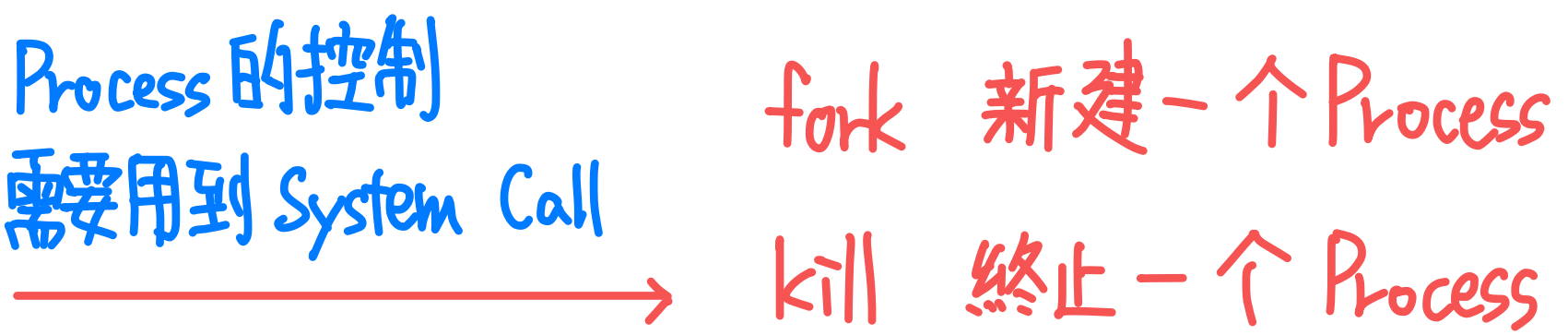
- Even simple programs may make **heavy use** of the operating system
 - Systems execute thousand of system calls per second
- Most programmers never see this level of detail
- Typically, **application developers** design programs according to an **application programming interface (API)**

System Call Implementation

- Typically, a **number (i)** associated with each system call
 - https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/
 - **System-call interface** maintains a **table** indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)



Types of System Calls

- Process Control 
 - create process, terminate process
 - load, execute
 - get process attributes, set process attributes
 - wait event, signal event
 - allocate and free memory
 - lock - `acquire_lock()`, `release_lock()`
- fork 新建一个 Process*
- kill 终止一个 Process*



One Developer Army

@OneDeveloperArmy

Sometimes, I wonder if non-tech friends wonder about my search history ...

Google

remove child from parent with fork

All Images Videos News More Settings Tools

About 7,350,000 results (0.50 seconds)

how to kill child of fork? - Stack Overflow
<https://stackoverflow.com/questions/13273836/how-to-kill-child-of-fork> ▾
3 answers
Nov 8, 2012 - I have the following code which create a child fork. And I want to kill the child before it finish its execution in the parent. how to do it? #include ...

Fork and child/parent processes	1 answer	16 May 2017
pid from fork() never goes to parent, only child	1 answer	12 Aug 2016
How to use fork() to daemonize a child process ...	4 answers	27 Jan 2012
Are child processes created with fork() automatically ...	1 answer	18 Nov 2011

More results from stackoverflow.com

5:41 AM · 11 Jun 19 · Twitter Web App

108 Retweets 377 Likes



LaurieWired

@lauriewired

Follow

OS internals books are wild...

Contents

- 9.4 Process Primitives
 - 9.4.1 Having Children
 - 9.4.2 Watching Your Children Die
 - 9.4.3 Running New Programs
 - 9.4.4 A Bit of History: `vfork()`
 - 9.4.5 Killing Yourself
 - 9.4.6 Killing Others
 - 9.4.7 Dumping Core
- 9.5 Simple Children

4:00 am · 07 Jul 23 · 1.8M Views

Types of System Calls

- File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes
 - logically attach or detach devices

Types of System Calls

- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages
 - transfer status information
 - attach and detach remote devices

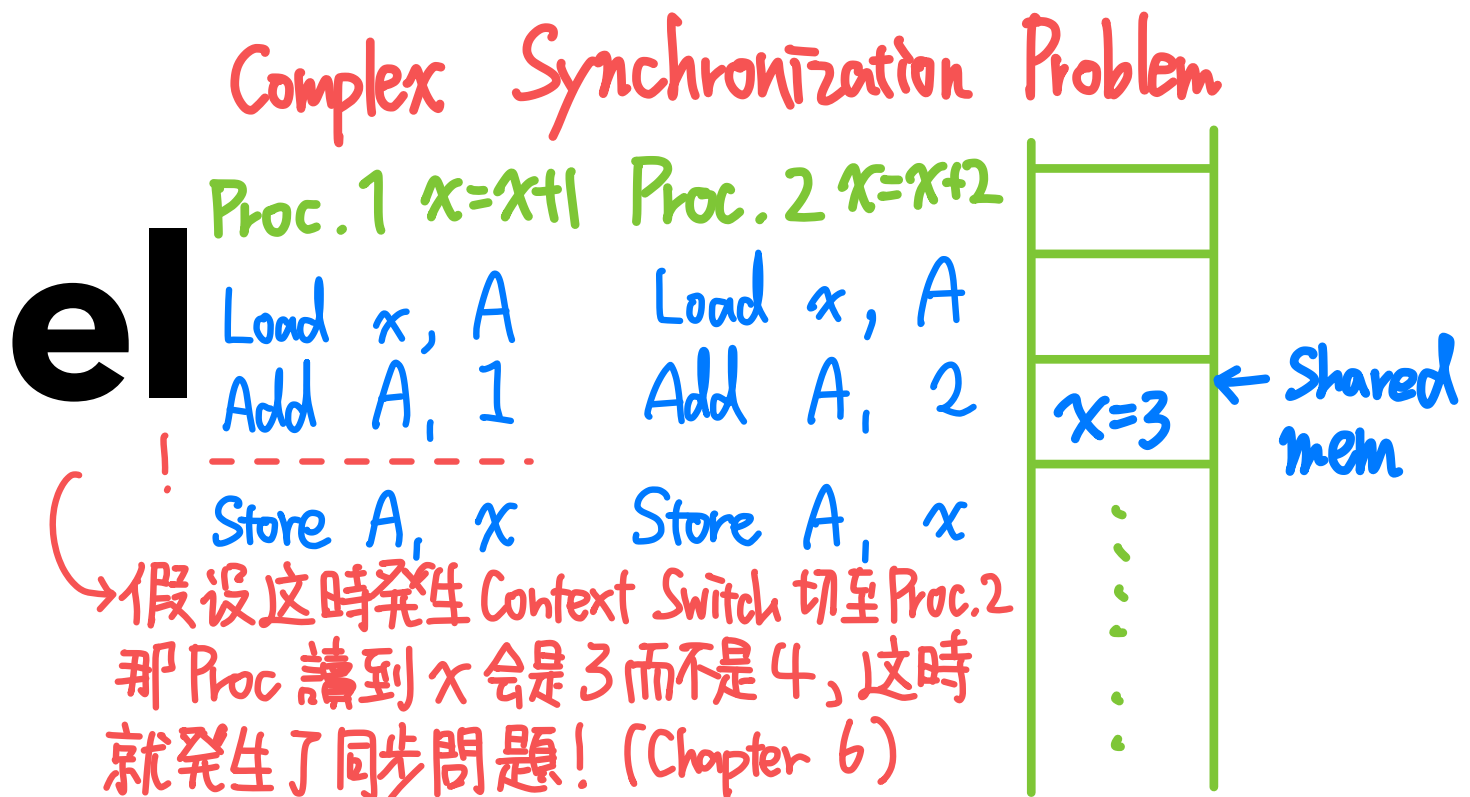
Types of System Calls

- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages
 - transfer status information
 - attach and detach remote devices
- Protection
 - get and set file permissions

Communication

- Inter-Process Communication (IPC)
 - Shared Memory Model (兩 Process 共同使用的資源, 可互相交換資訊) (有同步問題)
 - Message-Passing Model (類似 Network 訊息交換)

Shared Memory Model



- Advantages:
 - **Fast Communication**: The architecture enables all processes to directly access shared memory, resulting in significantly faster communication speeds.
 - **Efficient for Large Data Transfers**: Large data structures can be transferred between processes in complete blocks without requiring redundant copying into each process's memory space.
- Disadvantage:
 - Chapter 6
 - **Complex Synchronization**: Processes must implement specialized coordination tools like semaphores or mutex locks to prevent race conditions, adding complexity to the programming model.
 - **Security Risks**: The shared memory approach introduces significant security concerns, as any process with access privileges can potentially read or modify the shared data, creating exposure to unauthorized access.
 - **Single-Machine Constraint**: This mechanism cannot extend beyond individual computers in distributed systems, limiting its applicability to processes operating within the same physical machine.

Message Passing Model

- Advantages:
 - **No Synchronization Required**: Since processes do not directly share memory spaces, synchronization concerns are fundamentally eliminated, removing the need for complex locking mechanisms.
 - **Distributed Computing Compatibility**: This model naturally extends to inter-machine process communication, making it the preferred choice for distributed system architectures.

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- **System Services**
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

System Services

- **System services**, also known as **system utilities**, provide a convenient environment for program development and execution
 - File management (e.g. windows 檔案總管 / macOS Finder)
 - Status information
 - File modification (text editors)
 - Programming-language support
 - Compilers, assemblers, debuggers, etc.
 - Program loading and execution
 - Communications
 - Background services

Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- **Linkers and Loaders**
- Operating-System Design and Implementation
- Operating-System Structure

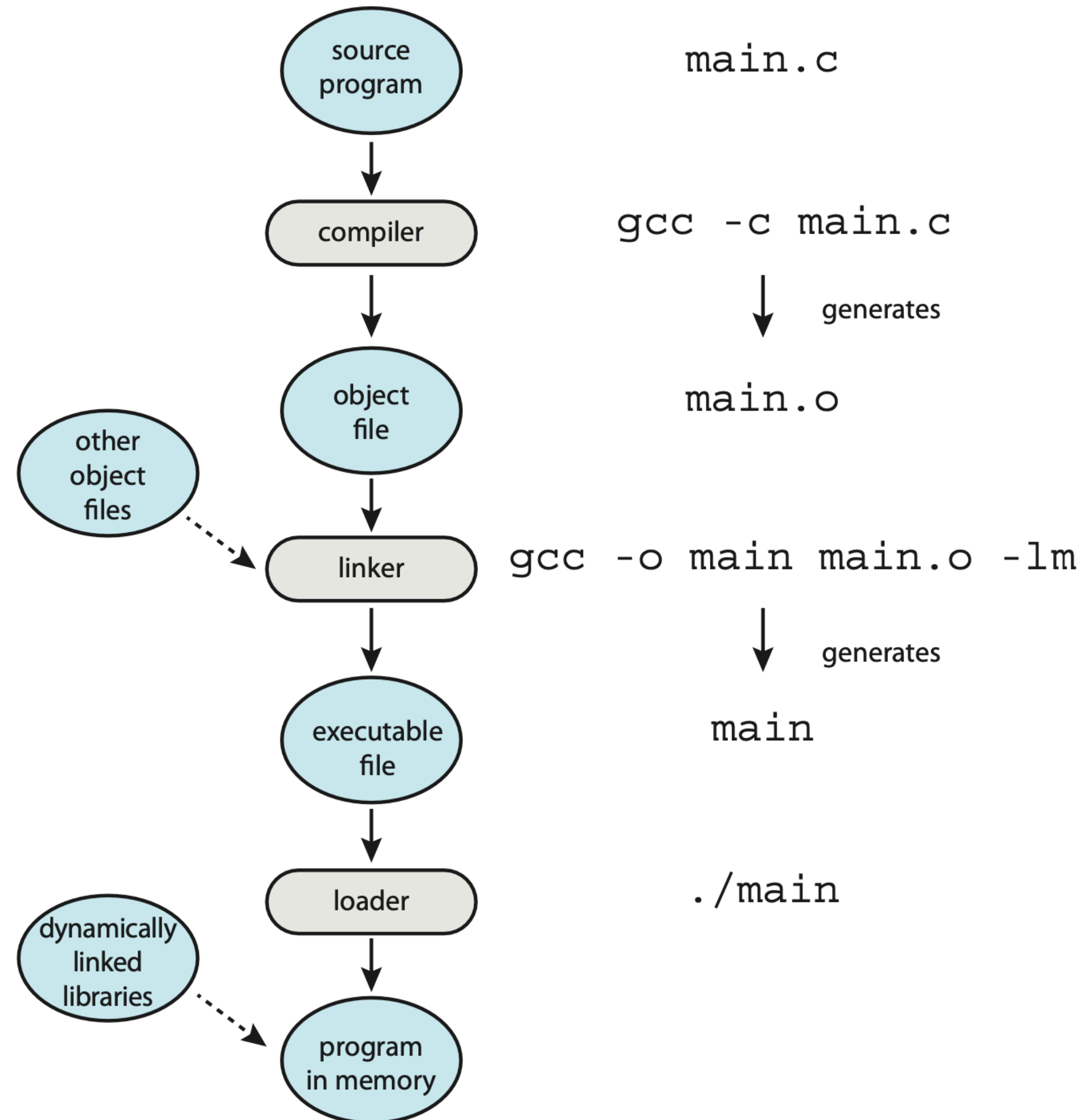
Linkers and Loaders

先編譯成 object 檔

obj 檔

- Source files (e.g., hello.c) are compiled into object files (e.g., hello.o) that are designed to be loaded into any physical memory location, a format known as an relocatable object file
- The linker combines these relocatable object files into a single binary executable file (e.g., hello.exe)
Linker 將 Library 函式的機器碼和編出來的 obj (機器碼) 連結成可執行文件
- A loader is used to load the binary executable file into memory
- An activity associated with linking and loading is relocation
- Dynamically Linked Libraries (DLLs, .dll), shared library (.so)
 - The library is conditionally linked and is loaded if it is required during program run time
動態連結庫, Program 在執行 (Run Time) 時若有需要才用 Linker 將 DLL 連結起來, 節省 mem.

Linkers and Loaders



Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

Design Goals

作業系統的設計和實作(理想)

- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- User goals and System goals
 - User goals
 - operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - System goals
 - operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

Mechanism & Policy

*重要!

- Important principle to separate
 - 演算法(或一種策略) (e.g. context switch 下一个 Proc 要執行誰?)
 - **Policy**: What will be done?
 - **Mechanism**: How to do it?
 - 一種 OS 的機制 (e.g. context switch, 要有該機制才能實現 Time Sharing)
- In operating system design, Policy and Mechanism are two core concepts, and their separation represents a fundamental principle in modern OS architecture.

Mechanism & Policy

- Basic Definitions
 - Mechanism
 - The underlying methods and tools that implement specific functions, answering the question of "how to do it." These are the **basic operations and functionalities** provided by the operating system. (由OS提供最基本的操作&函式)
 - Policy
 - The **high-level decisions** that determine when and how to use these mechanisms, answering the questions of "what to do" and "when to do it."

Mechanism & Policy

- Relationship and Distinctions
 - Separation Principle
 - Good operating system design **separates mechanisms from policies**
 - Mechanisms should be **flexible enough** to support multiple different policies
 - Policies can be adjusted according to system requirements and environmental changes **without altering the underlying mechanisms**
- Hierarchical Relationship:
 - Mechanisms exist at a lower level, providing basic functionality
 - Policies operate at a higher level, utilizing these mechanisms to achieve system objectives
- Flexibility and Scalability
 - Policies can be changed without affecting mechanisms
 - Enables the system to adapt to different usage environments and requirements

／ 過於理想化

Mechanism & Policy

- Illustrative Examples
 - Process Scheduling
 - Mechanism: Capability for context switching, process blocking/waking, priority setting
 - Policy: When to switch processes, which scheduling algorithm to use (FIFO, Round Robin, Multilevel Feedback Queue)
 - Memory Management
 - Mechanism: Paging, segmentation, virtual memory mapping
 - Policy: Page replacement algorithms (LRU, FIFO), when to perform swapping

Implementation

- Much variation
 - Early OSes in assembly language
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
- Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
 - More high-level language easier to **port** to other hardware
 - But slower

Outline

- Operating-System Services
- User and Operating-System Interface
- System Calls
- System Services
- Linkers and Loaders
- Operating-System Design and Implementation
- Operating-System Structure

Operating-System Structure

- A common approach is to partition the task into **small components**, or **modules**, rather than have **one single system**
- You may use a similar approach when you **structure your programs**: rather than placing all of your code in the `main()` function, you instead separate logic into a number of functions, clearly articulate parameters and return values, and then call those functions from `main()`.

Monolithic Structure

整體式架構

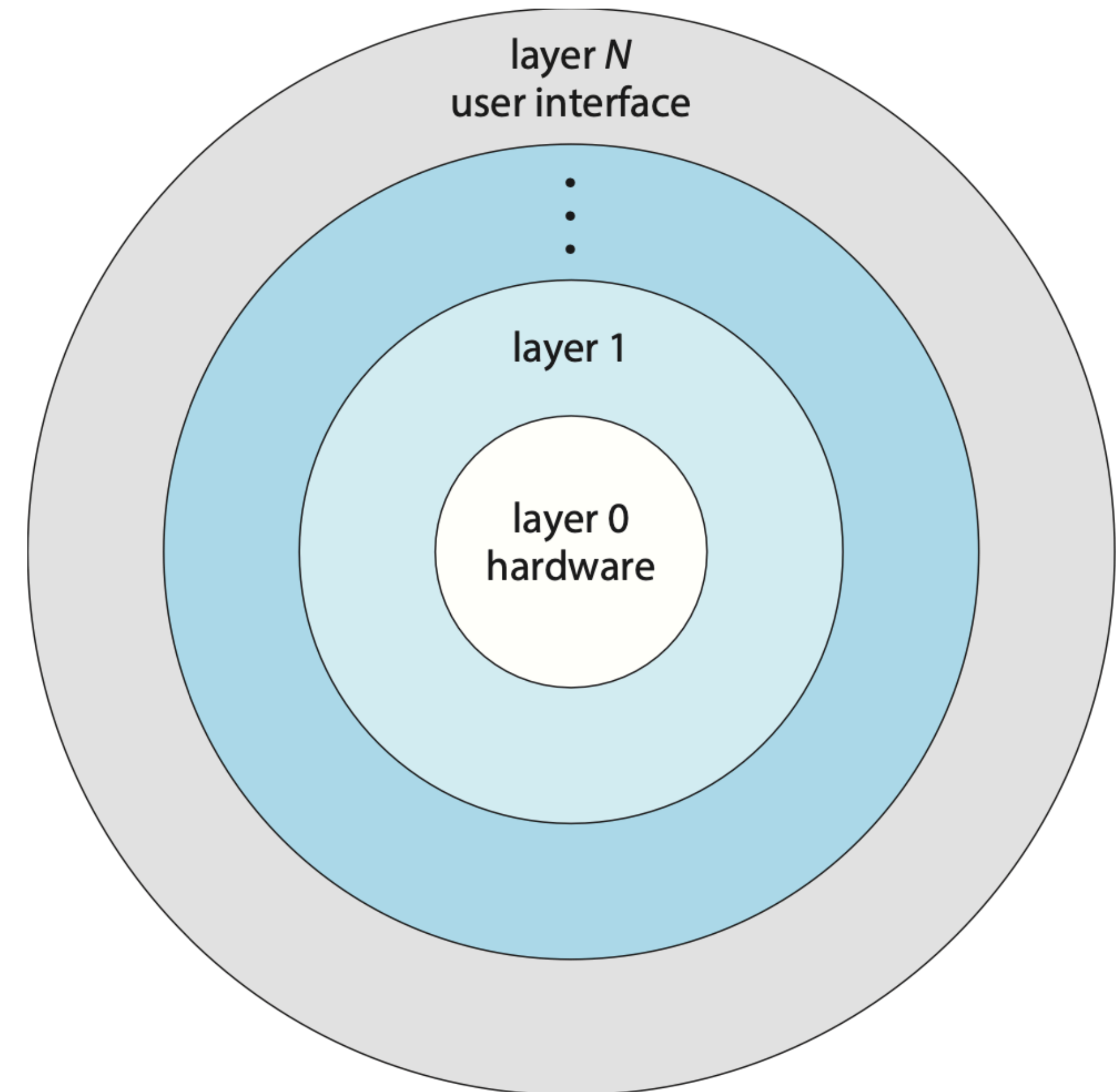
- The **simplest** structure for organizing an operating system is **no structure at all**
 - That is, place all of the functionality of the kernel into a single, static binary file that runs in a single address space
 - This approach—known as a **monolithic structure** (整體式)—is a common technique for designing operating systems
 - Original UNIX operating system
- Disadvantage: they are **difficult to implement and extend**
 - A **tightly coupled** system because changes to one part of the system can have wide-ranging effects on other parts
- Advantage: there is very little overhead in the system-call interface, and communication within the kernel is fast

Layered Structure

- A loosely coupled system
 - Flexibility
- Layered structure
 - The operating system is broken into a number of layers (levels)
 - The bottom layer (layer 0) is the hardware
 - The highest (layer N) is the user interface

* 每一層都單獨設計

* 實務上不太有作業系統採用此設計

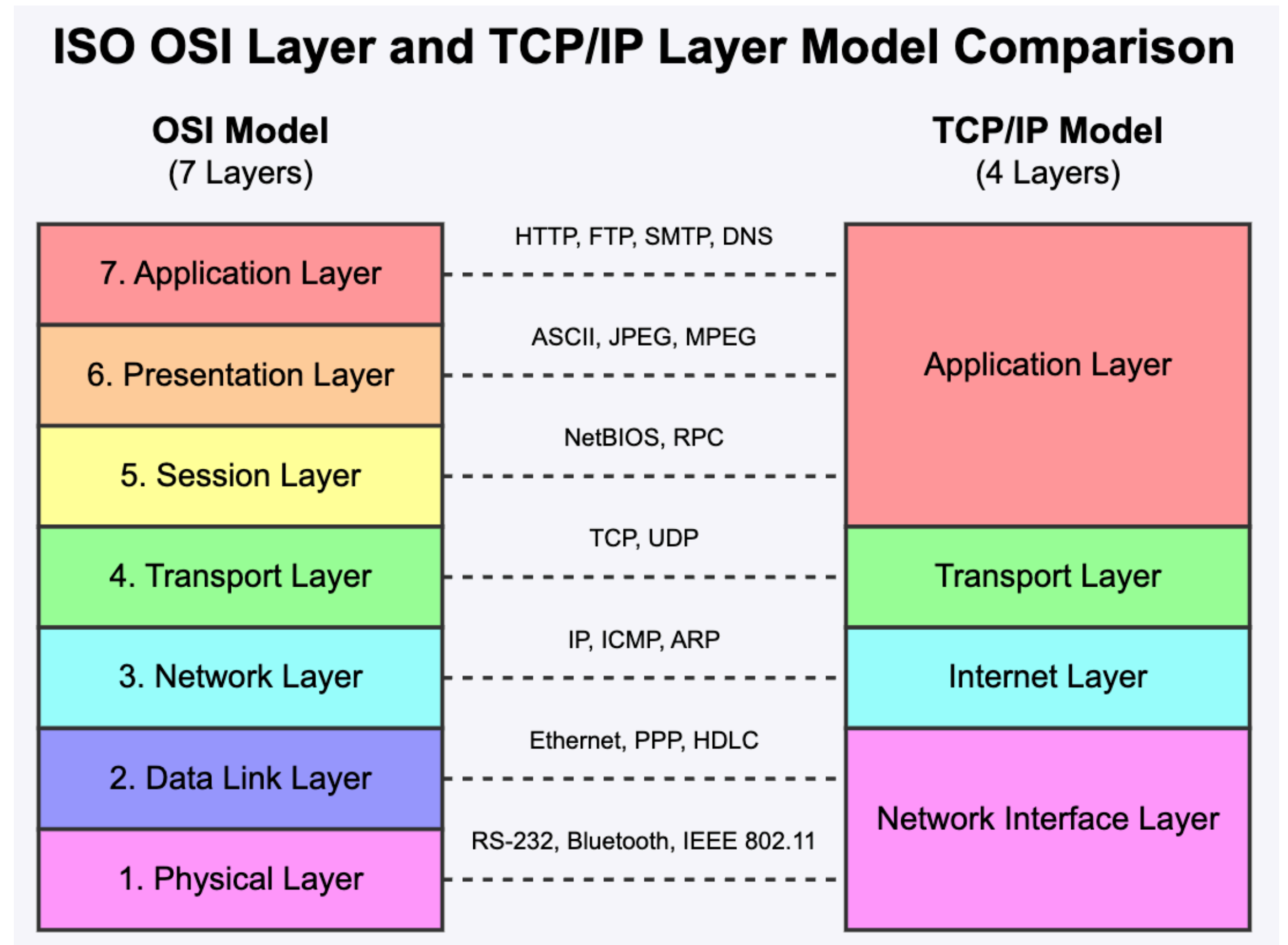


Layered Structure

- The main advantage of the layered approach is simplicity of construction and debugging
 - Each layer is implemented only with operations **provided by lower-level layers**
 - A layer does not need to know how these operations are implemented
 - It needs to know only what these operations do

Layered Structure

- The main advantage of the layered approach is simplicity of construction and debugging
- Each layer is implemented only with operations **provided by lower-level layers**
- A layer does not need to know how these operations are implemented
- It needs to know only what these operations do
- **Layered systems** have been successfully used in **computer networks**



Layered Structure

- Relatively few operating systems use a pure layered approach
 - The challenges of appropriately defining the functionality of each layer
 - The overall performance of such systems is poor due to the overhead of requiring a user program to traverse through multiple layers to obtain an operating-system service

Microkernels

微核心架構 (理想)

- Microkernel-based operating systems take the separation of mechanism and policy to one extreme by implementing **a basic set of primitive building blocks** 作業系統的一部分 類似 Application
- Moves as much from the kernel into user space
- Mach Operating System (Carnegie Mellon University)
- Communication takes place between user modules using message passing
- Benefits
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Detriments
 - Performance overhead of user space to kernel space communication

Microkernels

In microkernel architectures, the placement of CPU Scheduling Algorithms is an important design decision, and different microkernel systems handle this differently:

Two Main Design Approaches

1. Scheduling Algorithm in Kernel

Complete scheduler in kernel space:

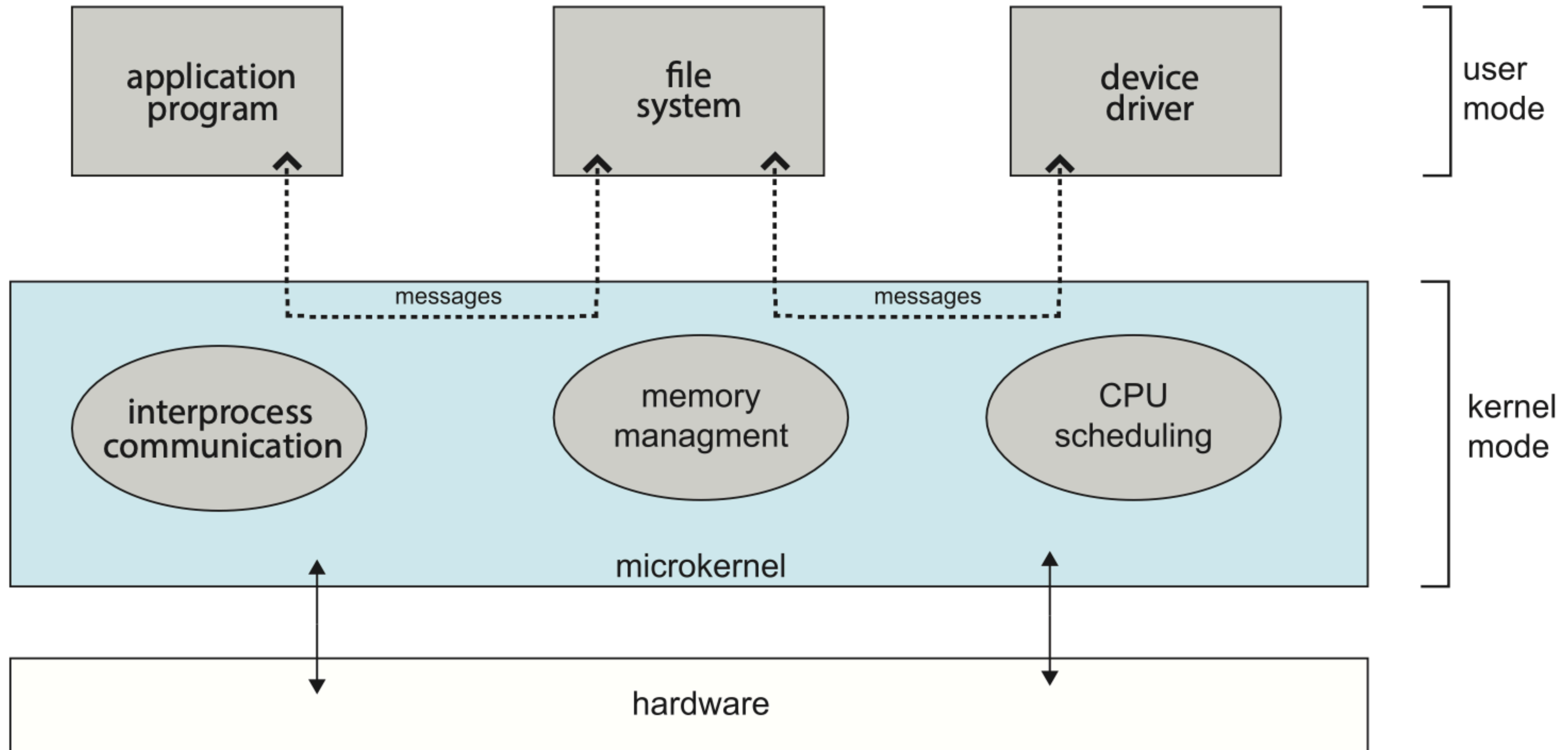
- Systems like **QNX, L4** adopt this approach
- Kernel includes complete scheduling algorithms (Round Robin, Priority-based, etc.)
- **Advantages:** Fast scheduling decisions, efficient context switching
- **Disadvantages:** Increases kernel complexity

2. Only Basic Mechanisms in Kernel

Minimal microkernel design:

- Kernel only provides basic **context switch mechanisms**
- Scheduling policy determined by **user-space scheduler**
- Example: Some research microkernel implementations

Architecture of a typical microkernel



Loadable Kernel Modules

(Linux, Windows 採用此設計)

- Loadable Kernel Modules (LKMs)
 - The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running
- Reference: <https://haogroot.com/2020/12/15/linux-kernel-module/>

Loadable Kernel Modules

- Loadable Kernel Modules (LKMs)
 - The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running
 - Many modern operating systems implement loadable kernel modules
 - Each core component is separate
 - Each talks to the others over [known interfaces](#)
 - Each is loadable as needed within the kernel
- Reference: <https://haogroot.com/2020/12/15/linux-kernel-module/>

Hybrid Systems

- Most modern operating systems are actually not one pure model
 - Hybrid combines multiple approaches to address performance, security, usability needs
 - **Linux** and Solaris kernels in kernel address space, so **monolithic**, plus **modular for dynamic loading** of functionality
 - Windows mostly monolithic, plus microkernel for different subsystem personalities

大部分作業系統都还是整体式设计，並同時搭配動態載入的功能。